MANIPULACIÓN DE DATOS

UNIDAD 2 - Parte 1

Fundamentos de Ciencia de Datos 2026

SAY IT AGAIN DEXTER
I'M A DATA SCIENTIST

¿Qué son los DATOS?

Los datos son **piezas de información** de algún tema o área en particular.

Pueden venir en tres formas diferentes según su estructura:

- Estructurados
- No estructurados
- Semiestructurados

DATOS ESTRUCTURADOS

Los datos estructurados tienen un **formato estandarizado o definido** que permite tanto al software como a las personas acceder a la información de una manera eficiente. Por lo general, suelen organizarse **en forma de tablas con filas y columnas**.

Cada campo o atributo tiene un tipo de dato y un formato claro.

DATOS ESTRUCTURADOS - características

Atributos definibles

Los datos estructurados tienen los mismos atributos para los diferentes registros.

DATOS ESTRUCTURADOS - características

Atributos relacionales

Las tablas de datos estructurados tienen información común que permite vincular entre sí diferentes conjuntos de datos.

id	name	age
1	Jim	28
2	Pam	26
3	Michael	42

id	subject	Teacher
1	Languages	John Jones
2	Track	Wally West
3	Swimming	Arthur Curry
4	Computers	Victor Stone

student_id	subject_id	grade
2	1	98
1	2	100
1	4	75
3	3	60
2	4	76
3	2	88

DATOS ESTRUCTURADOS - características

Almacenamiento

Se pueden almacenar datos estructurados de manera conveniente en una **base de datos relacional** y administrarlos y consultarlos en forma sencilla con lenguajes como SQL (*Structured Query Language*).

DATOS NO ESTRUCTURADOS

Los datos no estructurados son información que **no posee un modelo de datos predefinido ni un esquema fijo**, y cuya organización interna **no sigue una estructura tabular.** Por esta razón, no pueden representarse naturalmente mediante filas y columnas.

Ejemplos: correos electrónicos, *chats*, documentos de texto, imágenes, audio y video.

DIFERENCIAS

Facilidad de análisis Es **mucho más difícil organizar, limpiar y analizar datos que no tengan un modelo de datos predefinido** y muy pocas herramientas disponibles en el mercado pueden hacer esto.

Capacidad de búsqueda Es **fácil hacer búsquedas en los datos estructurados** ya que siguen diversas reglas predefinidas.

Flexibilidad **Los datos no estructurados tienen menos restricciones en su estructura**, por lo que es más fácil agregar nueva información en comparación con un conjunto de datos estructurados.

DATOS SEMI-ESTRUCTURADOS

Representan una **categoría intermedia entre los datos estructurados y los no estructurados**. Tienen cierta organización interna, pero **no siguen un esquema rígido** como el de las tablas de una base de datos relacional.

Ejemplos típicos son los archivos **XML, YAML o JSON**, que utilizan etiquetas (*tags*) o pares clave-valor para dar estructura, pero permiten flexibilidad en el formato y en los tipos de datos que contienen.

XML (*Extensible Markup Language*)

Está basado en texto, lo que lo hace fácil de leer para humanos y computadoras.

El siguiente código representa el registro de una persona. **¿Qué información contiene?**

```
<Person Age="23">
    <FirstName>Quinn</FirstName>
    <LastName>Anderson</LastName>
    <Hobbies>
        <Hobby Type="Sports">Golf</Hobby>
        <Hobby Type="Leisure">Reading</Hobby>
        <Hobby Type="Leisure">Guitar</Hobby>
   </Hobbies>
</Person>
```

XML (*Extensible Markup Language*)

XML utiliza ***tags*** para darles forma a los datos, los cuales pueden ser:

- **Elementos**, como `<First Name>`.
- **Atributos**, como `Age='23'` .

A su vez, los elementos pueden tener elementos secundarios o hijos que permiten expresar relaciones, como `Hobby` dentro del elemento `Hobbies`.

JSON

Es un formato de archivo liviano que se utiliza para almacenar e intercambiar datos y se basa en el lenguaje de programación **JavaScript**.

Utiliza una **estructura clave-valor** para representar datos y soporta el manejo de listas y jerarquías.

JSON usa llaves `{}` para indicar la estructura de los datos.

JSON

Ejemplo conocido...

```
1  {
2      "firstName": "Quinn",
3      "lastName": "Anderson",
4      "age": "23",
5      "hobbies": [
6          { "type": "Sports", "value": "Golf" },
7          { "type": "Leisure", "value": "Reading" },
8          { "type": "Leisure", "value": "Guitar" }
9      ]
10 }
```

YAML

YAML Ain't Markup Language (YAML) es un lenguaje de serialización con una gran legibilidad.

La estructura de los datos se da por la **separación de las líneas** y la **indentación** (se elimina la dependencia de caracteres especiales).

```yaml
firstName: Quinn
lastName: Anderson
age: 23
hobbies:
    - type: Sports
      value: Golf
    - type: Leisure
      value: Reading
    - type: Leisure
      value: Guitar
```

XML vs. JSON vs. YAML

XML. Formato basado en etiquetas. Muy verboso y complejo para la legibilidad humana. Usado históricamente en la integración de sistemas.

JSON. Formato ligero basado en pares clave-valor y listas. Estándar actual para intercambio de datos en la web y APIs.

YAML. Formato basado en indentación y pares clave-valor muy legible para humanos. Ampliamente usado en archivos de configuración.

LA ERA DE LOS DATOS NO ESTRUCTURADOS

Se estima que **más del 80% de los datos generados a nivel mundial son *no estructurados***, y gran parte de ellos son datos textuales, como *e-mails*, posteos en redes sociales, artículos de noticias, etc.

DATOS TABULARES

Aunque la gran mayoría de los datos generados en el mundo real son no estructurados, en el análisis de datos es común encontrarnos con datos representados en un **formato tabular** (compuestos por filas y columnas).

Este será el tipo de datos con el que trabajaremos principalmente a lo largo de esta asignatura.

Para almacenar datos tabulares se pueden usar diferentes tipos de archivos: **.csv, .json, .txt, .html, .parquet.**

ARCHIVOS ORIENTADOS A FILAS/COLUMNAS

ORIENTADOS A FILAS

Los datos **se organizan en registros** y todos los datos asociados a un registro se guardan juntos en la memoria. Por lo tanto, realizar consultas sobre el valor de un atributo para diferentes registros resulta ineficiente ya que se debe cargar todo el registro con datos innecesarios.

ARCHIVOS ORIENTADOS A FILAS/COLUMNAS

ORIENTADOS A COLUMNAS

Los datos **se organizan por columna/campo/variable** y todos los datos de la columna se guardan juntos en la memoria. Por lo tanto, cuando queremos consultar los valores de una columna, sólo necesitamos cargar esa columna sin necesidad de leer todo el archivo.

Además, al ser todos los datos de una misma columna del mismo tipo, la compresión del archivo es mejor.

ARCHIVOS ORIENTADOS A FILAS/COLUMNAS

Supongamos que tenemos la siguiente tabla:

dni	nombre	apellido	anio_nacimiento
40576890	Pedro	Aguirre	1995
32492645	Julia	Martinez	1988

¿Qué forma tendrá el archivo si dicha información se guarda orientada a filas?

¿Y si se guarda orientada a columnas?

ARCHIVOS ORIENTADOS A FILAS/COLUMNAS

Si la info se guarda **orientada a filas**:

row	value
row 1	40576890
	Pedro
	Aguirre
	1995
row 2	32492645
	Julia
	Martinez
	1988

ARCHIVOS ORIENTADOS A FILAS/COLUMNAS

Si la info se guarda **orientada a columnas**:

column	value
dni	40576890
	32492645
nombre	Pedro
	Julia
apellido	Aguirre
	Martinez
año_nacimiento	1995
	1988

ARCHIVOS PARA ALMACENAMIENTO DE DATOS TABULARES

A continuación, veremos diferentes formatos de archivos para almacenar datos tabulares, con los que trabajaremos a lo largo del semestre:

- **.csv**
- **.json**
- **.txt**
- **.html**
- **.parquet**

CSV (*Comma-Separated Values*)

Los diferentes registros (las filas) se separan entre sí mediante saltos de líneas, mientras que los atributos/variables (las columnas) se separan usando la **coma** (también se pueden usar otros caracteres como el **punto y coma** o el ***tab***). Hoy en día es uno de los formatos más utilizados en el análisis de datos.

```
Name, Age, Gender
John, 25, Male
Jane, 30, Female
Bob, 40, Male
```

CSV (*Comma-Separated Values*)

Para tener en cuenta:

- Los archivos .csv son archivos poseen un formato de almacenamiento **orientado a filas**.
- Los archivos .csv **no guardan información acerca del tipo de dato** de su contenido puesto que todo se guarda en texto simple.

TXT

Es uno de los formatos más simples y ampliamente utilizados para el almacenamiento de datos.

Cuando se almacenan datos tabulares, los archivos TXT suelen ser muy similares a los CSV.

Aplicaciones: se utiliza comúnmente para almacenar grandes conjuntos de datos de texto, como documentos, transcripciones, registros de chat, mensajes de correo electrónico, etc. También se utiliza en el procesamiento de lenguaje natural (NLP) para almacenar y analizar conjuntos de texto, como textos médicos, noticias, redes sociales, entre otros.

APACHE PARQUET

Es un formato para el almacenamiento de datos tabulares **orientado a columnas** y está optimizado para **grandes cargas de datos**. Es de uso común en sistemas de procesamiento de *Big Data* basados en Hadoop**,** como Hive, Impala y Spark. Fue desarrollado por Cloudera y Twitter en 2013 como un proyecto de código abierto.

APACHE PARQUET

Parquet se basa en una representación de datos en **columnas comprimidas**, lo que lo hace muy eficiente para consultas analíticas que involucran grandes cantidades de datos. Es un formato popular para el procesamiento de *Big Data*.

HTML

El formato HTML (Lenguaje de Marcado de Hipertexto o *HyperText Markup Language*) es un lenguaje utilizado principalmente para crear páginas web y documentos de hipertexto.

En el campo de la ciencia de datos, se puede utilizar como un formato de almacenamiento de datos estructurados y no estructurados.

HTML

Este es un ejemplo de una tabla escrita en HTML:

```
<html>
    <head></head>
    <body>
        <table id="customers">
  <tbody>
        <tr>
        <th>Company</th>
        <th>Contact</th>
        <th>Country</th>
      </tr>
      <tr>
```

HTML

HTML (al igual que XML) utiliza *tags* para estructurar el contenido:

📌 **Elementos clave de la sintaxis:**

<table>: define el inicio y el fin de la tabla.
<tr> (*table row*): representa una fila.
<th> (*table header*): representa una celda de encabezado.
<td> (*table data*): representa una celda de datos.

A esto se lo llama **sintaxis de marcado**, porque las etiquetas "marcan" o delimitan parte del contenido.

REPASO DE PANDAS

PANDAS

PANDAS es una librería de Python para el análisis y manipulación de datos. Proporciona **estructuras de datos** eficientes para almacenar y organizar información, y un conjunto de **funciones** que permiten realizar una gran variedad de operaciones, como filtrar, transformar, agrupar o resumir datos, entre muchas otras.

ESTRUCTURAS DE DATOS BÁSICAS EN PANDAS

Pandas proporciona dos estructuras principales para trabajar con datos:

- `Series`: una serie de Pandas es una matriz unidimensional capaz de contener cualquier tipo de dato: números enteros, cadenas de texto, números decimales, objetos de Python, etc. Cada elemento de la serie posee un identificador único llamado **índice** (`index`).
- `DataFrame`: un DataFrame es una estructura bidimensional tabular formada por filas y columnas. Cada fila está identificada por un índice, y las distintas columnas pueden almacenar datos de diferente tipo.

TIPOS DE DATOS USUALES

En el trabajo con datos tabulares aparecen con frecuencia los siguientes tipos de datos:

- **`int`**: para representar valores enteros.
- **`float`**: para representar valores reales en coma flotante.
- **`str`**: sirve para representar texto.
- **`bool`**: para representar valores booleanos: `True` o `False`.
- **`NaN/None`**: para representar valores faltantes (ausentes o desconocidos).

LECTURA DE ARCHIVOS CON DATOS TABULARES

Pandas permite leer datos desde múltiples formatos de archivo y convertirlos directamente en `DataFrames`. La lectura de datos se realiza mediante funciones específicas para cada tipo de archivo, por ejemplo:

- `read_csv()`. Permite leer tanto archivos CSV como TXT.
- `read_excel()`. Permite leer tanto archivos .xlsx como .xls.
- `read_json()`.
- `read_parquet()`

`object` vs. `str`

En pandas, las columnas que contienen valores de texto se representan como el tipo de datos **`object`** en lugar del tipo de datos **`str`** de Python. Esto se debe a que **`str`** es un tipo de datos específico en Python que sólo puede contener caracteres alfanuméricos y algunos caracteres especiales, mientras que **`object`** puede contener cualquier tipo de objeto de Python, incluyendo cadenas de texto.

Además, las columnas de texto en un **DataFrame** de pandas pueden contener valores faltantes (por ejemplo, NaN), y **`object`** es un tipo de datos compatible con este tipo de datos.

MANEJO DE FECHAS

MANEJO DE FECHAS

El módulo `datetime` de Python nos provee clases para manipular fechas y horas. Los objetos de fecha y hora pueden ser categorizados como **aware** o **naive**, dependiendo de si incluyen o no información sobre el huso horario.

MANEJO DE FECHAS

Un **objeto de fecha y hora de tipo *naive*** no contiene información sobre la zona horaria. Representa una fecha y una hora determinadas, pero no queda especificado a qué huso horario se refiere.

Un **objeto de fecha y hora de tipo *aware*** contiene información explícita sobre la zona horaria, lo que permite representar un instante específico en el tiempo de forma inequívoca.

MANEJO DE FECHAS - Ejemplo

Por ejemplo, podemos elegir representar la fecha y la hora del primer partido de Argentina en la Copa Mundial FIFA 2026, frente a Argelia en Kansas City.

Objeto de fecha y hora de tipo *naive*

```
from datetime import datetime

partido_naive = datetime(2026, 6, 16, 22, 0, 0)
print(partido_naive)
```

```
2026-06-16 22:00:00
```

MANEJO DE FECHAS - Ejemplo

Objeto de fecha y hora de tipo *aware*

Para trabajar con este tipo de objetos es habitual utilizar la librería **`pytz`**, que provee una base completa de husos horarios.

```
import pytz

zona_arg = pytz.timezone('America/Argentina/Buenos_Aires')
partido_aware = zona_arg.localize(datetime(2026, 6, 16, 22, 0, 0))

print(partido_aware)
```

```
2026-06-16 22:00:00-03:00
```

En este caso, el objeto de fecha y hora **contiene información explícita sobre la zona horaria de Argentina.**

🤪 ¡Manos a la obra! - PARTE I

- Leé el archivo `lista_personas.csv`.
- ¿Cuántas filas y columnas tiene el dataset?
- ¿Hay alguna columna que contenga datos faltantes?
- Observá los nombres de las columnas. ¿Detectás alguna inconsistencia?
- ¿Qué tipo de dato contiene cada columna? ¿Es el esperado en cada caso?
- ¿Cuántos nombres diferentes de personas hay en el dataset? ¿Observás algún error?

MÉTODOS ÚTILES

`.rename()`. Útil para renombrar columnas o índices.

```
df.rename(columns = {'nombre_viejo':'nombre_nuevo'})
```

`.str.replace()`. Útil para reemplazar caracteres o cadenas dentro de una columna de texto.

```
df['col'].str.replace('str_a_reemplazar', 'str_nuevo')
```

`.str.strip()`. Elimina espacios en blanco al inicio y al final de las cadenas.

```
df['col'].str.strip()
```

`.str.lower()`/**`.str.upper()`**. Convierte cadenas de texto a minúsculas o mayúsculas, respectivamente.

`.astype(tipo)`. Convierte el tipo de dato de una columna.

MÉTODOS ÚTILES

`.isnull()` / **`isna()`** / **`.notnull()`**. Son útiles para detectar valores faltantes. Aplicados sobre una columna de un DataFrame, devuelven una Serie booleana.

`.unique()`. Devuelve un *array* con los valores únicos presentes en una columna.

`.value_counts()`. Devuelve una Serie con las frecuencias de los valores únicos en una columna.

```
df['col'].value_counts()
```

CONVERSIÓN CON `pd.to_datetime()`

En muchos conjuntos de datos, las fechas y horas aparecen almacenadas como texto. La función `pd.to_datetime()` permite convertir estos valores en objetos de tipo fecha-hora, que luego pueden analizarse y manipularse fácilmente.

Una vez convertidas, las fechas permiten, por ejemplo:

- ordenar observaciones en el tiempo
- extraer componentes (año, mes, día, hora)
- calcular diferencias entre fechas

CONVERSIÓN CON `pd.to_datetime()`

Supongamos un conjunto de datos con la fecha de una compra:

```
import pandas as pd

df = pd.DataFrame({"fecha": ["2024-03-10", "2024-03-12", "2024-03-15"], "m

df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3 entries, 0 to 2
Data columns (total 2 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   fecha   3 non-null      object
 1   monto   3 non-null      int64
dtypes: int64(1), object(1)
memory usage: 180.0+ bytes
```

Inicialmente la columna “fecha” es de tipo `object`.

CONVERSIÓN CON `pd.to_datetime()`

```
df["fecha"] = pd.to_datetime(df["fecha"])

df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3 entries, 0 to 2
Data columns (total 2 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   fecha   3 non-null      datetime64[ns]
 1   monto   3 non-null      int64
dtypes: datetime64[ns](1), int64(1)
memory usage: 180.0 bytes
```

Después de la conversión, la columna pasa a ser de tipo `datetime` y puede utilizarse para operaciones temporales.

CONVERSIÓN CON `pd.to_datetime()`

Cuando las fechas no están en un formato estándar o pueden interpretarse de más de una forma, es conveniente especificar cómo deben leerse mediante algunos parámetros de `pd.to_datetime()`. Por ejemplo:

- El parámetro **`dayfirst = True`** indica que el día aparece antes que el mes en la cadena de texto.
- También es posible indicar exactamente cómo está escrita la fecha mediante el argumento **`format`**. Por ejemplo: `format = '%d/%m/%Y'` indica que la fecha está escrita en la forma **DÍA/MES/AÑO CON CUATRO DÍGITOS**.

🤪 ¡Manos a la obra! - PARTE II

Considerando el dataset `lista_personas.csv`:

- ¿Quién es la persona de mayor edad entre las personas del dataset?
- Extraer el mes de nacimiento de cada persona en una nueva columna. ¿Cuál fue el mes con mayor cantidad de nacimientos?

SOBRE `datetime64`

`datetime64` es un tipo de datos numérico que se representa internamente como un **número entero de 64 bits.**

Cada unidad de fecha y hora (año, mes, día, hora, minuto, segundo, nanosegundo) se convierte en un número entero que representa la cantidad de esa unidad desde una fecha de referencia, que es el **1 de enero de 1970**. Esta fecha de referencia se utiliza como base para el cálculo de todas las demás fechas y horas.

SOBRE `datetime64`

La precisión de `datetime64` se puede controlar mediante los modificadores de unidades de tiempo. Por ejemplo, `datetime64[s]` representa una fecha y hora con precisión de segundos, `datetime64[ms]` con precisión de milisegundos y `datetime64[us]` con precisión de microsegundos.

So what's your idea of a perfect date?
YYYY-MM-DD
I find other formats a bit confusing.
Pun hub